Question 1 – Part C(a)
Gradient-Based Feature Attribution (Analytical)

Network Architecture

We consider a two-hidden-layer multilayer perceptron (MLP):

$$
\begin{aligned}
z_1 &= W_1 x + b_1 \\
h_1 &= g(z_1) \\
z_2 &= W_2 h_1 + b_2 \\
h_2 &= g(z_2) \\
z_3 &= W_3 h_2 + b_3 \\
\hat{y} &= \text{softmax}(z_3)
\end{aligned}
$$

The loss function is cross-entropy:

$$L = -\sum_c y_c \log(\hat{y}_c)$$

Our objective is to compute the gradient of the loss with respect to the input vector:

$$\frac{\partial L}{\partial x}$$

This quantity tells us how sensitive the loss is to small changes in each input feature.

Step-by-Step Derivation

Step 1: Gradient at the Output Layer

For softmax combined with cross-entropy loss, the derivative simplifies to:

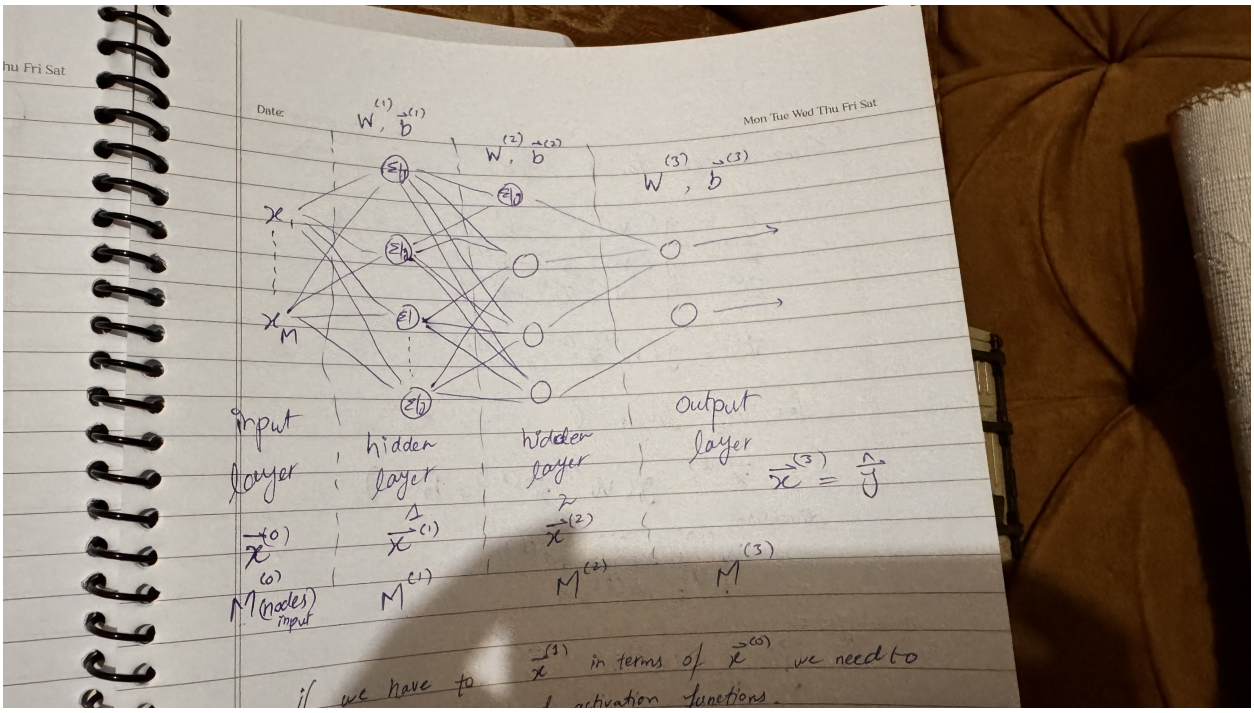

Figure 1: 2 Hidden Layers NN

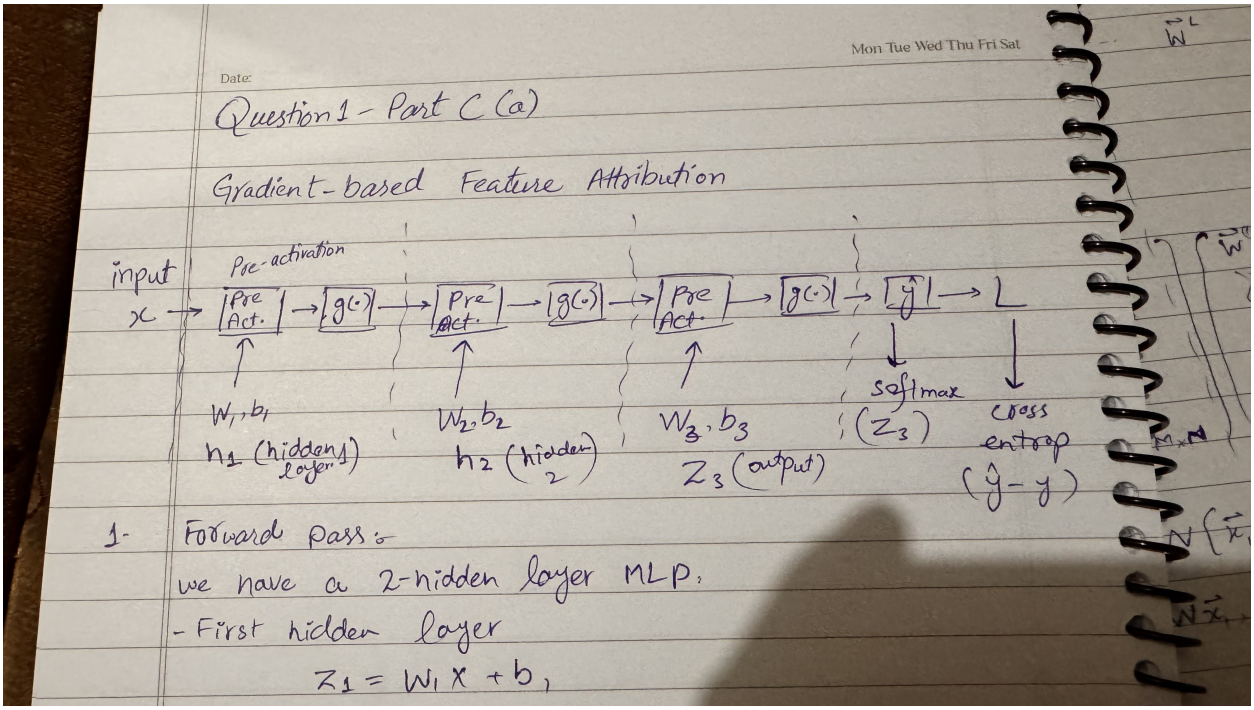

Figure 2: Zoomed Out NN

$$\frac{\partial L}{\partial z_3} = \hat{y} - y$$

We define:

$$\delta_3 = \hat{y} - y$$

This serves as the starting point for backpropagation.

Step 2: Backpropagation to the Second Hidden Layer

Since:

$$z_3 = W_3 h_2 + b_3$$

we obtain:

$$\frac{\partial L}{\partial h_2} = W_3^T \delta_3$$

Because $h_2 = g(z_2)$, we apply the activation derivative:

$$\delta_2 = \frac{\partial L}{\partial z_2} = (W_3^T \delta_3) \odot g'(z_2)$$

Step 3: Backpropagation to the First Hidden Layer

Since:

$$z_2 = W_2 h_1 + b_2$$

we compute:

$$\frac{\partial L}{\partial h_1} = W_2^T \delta_2$$

Applying the activation derivative:

$$\delta_1 = \frac{\partial L}{\partial z_1} = (W_2^T \delta_2) \odot g'(z_1)$$

Step 4: Gradient with Respect to Input

Finally, since:

$$z_1 = W_1 x + b_1$$

we obtain:

$$\frac{\partial L}{\partial x} = W_1^T \delta_1$$

Final Result

$$\delta_3 = \hat{y} - y$$

$$\delta_2 = (W_3^T \delta_3) \odot g'(z_2)$$

$$\delta_1 = (W_2^T \delta_2) \odot g'(z_1)$$

$$\boxed{\frac{\partial L}{\partial x} = W_1^T \delta_1}$$

This expression provides a gradient value for each input feature.

Gradient-Based Feature Attribution Algorithm

Algorithm 1 Gradient-Based Feature Attribution

Initialize $I = 0$ (dimension d)
for each training sample $(x^{(n)}, y^{(n)})$ **do**
    Perform forward pass to compute $\hat{y}$
    $\delta_3 = \hat{y} - y$
    $\delta_2 = (W_3^T \delta_3) \odot g'(z_2)$
    $\delta_1 = (W_2^T \delta_2) \odot g'(z_1)$
    $grad\_x = W_1^T \delta_1$
    $I = I + |grad\_x|$
end for
$I = I/N$
Rank features in descending order of I

Why $|\partial L/\partial x_i|$ is Meaningful

The gradient represents the direction of steepest change of the loss function. If $|\partial L/\partial x_i|$ is large, a small perturbation in feature x_i causes a significant change in the loss. This indicates that the model relies strongly on that feature for prediction.

If the gradient magnitude is small, the model is relatively insensitive to that feature near the current input.

By averaging the absolute gradient magnitude across multiple samples, we obtain a stable and principled estimate of feature importance grounded in optimization theory.